

STM32 按键控制 LED：实战排错与按键消抖知识整理

2026-08-11 | 工程：key (STM32F103C8T6 / Blue Pill) | STM32CubeIDE + HAL 库

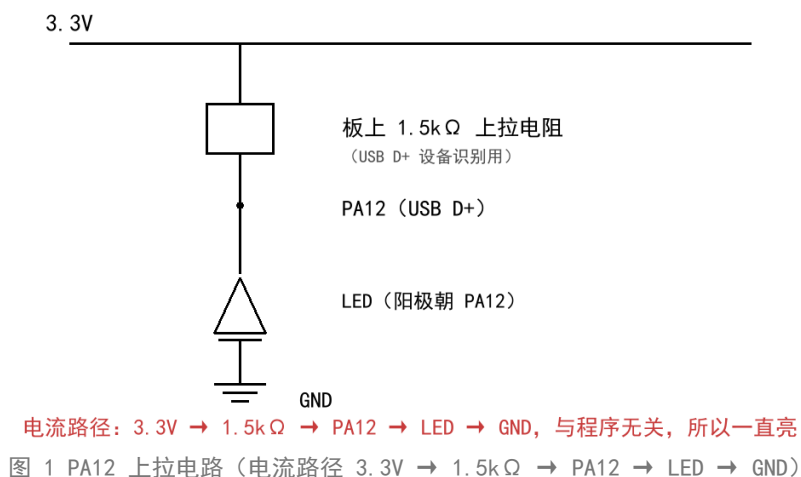
一、本次排错回顾

1.1 现象一：没配置 PA12，但接在 PA12 的 LED 一直亮

原因：PA12 就是 USB 的 D+ 引脚。Blue Pill 板上焊了 $1.5k\Omega$ 上拉电阻到 3.3V (USB 全速设备识别用)，引脚被硬件拉高。只要 LED 按「PA12 → 限流电阻 → LED → GND」接线，电流就会从 3.3V 经上拉电阻流过 LED，所以一直亮，与程序无关。

结论：不要用 PA12 接 LED；若必须用，改成「3.3V → 电阻 → LED → PA12」低电平点亮接法，并由程序主动把 PA12 拉低。

为什么 PA12 的 LED 一直亮 (板上 USB 上拉)



1.2 现象二：按键按下没反应

原因：面包板中间凹槽左右两排不导通。按键两个脚一个插凹槽左边、一个插右边，等于没接上，PA3 一直收不到信号。

检查方法：万用量表 PA3 对地，按下时应为 3.3V；或在 CubeIDE 调试里用 Live Expressions 看 GPIOA->IDR 的 bit3 是否从 0 变 1。

1.3 现象三：快速按键变成「按下亮、松开灭」

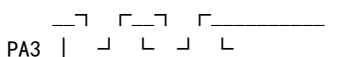
原因：延时后没有二次确认就翻转。松手瞬间触点会抖动，引脚短暂弹回高电平，外层 if 再次进入并再次翻转，于是每次按下翻转亮、松手又被翻转灭，看起来 LED 在跟随按键。

修复：延时后再次读取确认仍是按下才翻转；松手后再延时消抖，防止松手抖动再次触发。

二、按键为什么需要消抖

机械按键按下/松开的瞬间，金属触点会快速接通-断开-接通多次，产生「抖动」，一般持续 5~20ms。如果不处理，一次按键会被 CPU 读成多次，导致 LED 翻转多次、计数器多加等。

按下瞬间：



┌——抖动——┐ 稳定按下
松开瞬间同样会有一次抖动。

三、消抖方案一：延时消抖

原理：第一次读到「按下」后，先延时 20ms（抖动通常小于 20ms），再读一次。如果还是按下，说明是稳定电平，不是毛刺。

```
if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_3) == GPIO_PIN_SET) // 第一次读到按下
{
    HAL_Delay(20); // 等抖动平息
    if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_3) == GPIO_PIN_SET) // 再次确认
    {
        // 确认是真按下
    }
}
```

优点：简单直观。缺点：HAL_Delay 阻塞 20ms；只二次确认一次；无法区分长按/双击。

四、消抖方案二：状态机消抖（重点）

原理：不阻塞等待，而是固定周期采样（如每 1ms 一次），用计数器累计「连续读到相同电平」的次数，连续 N 次（例如 5 次）才确认状态变化。按下和松开都消抖，一次按键只产生一次事件。

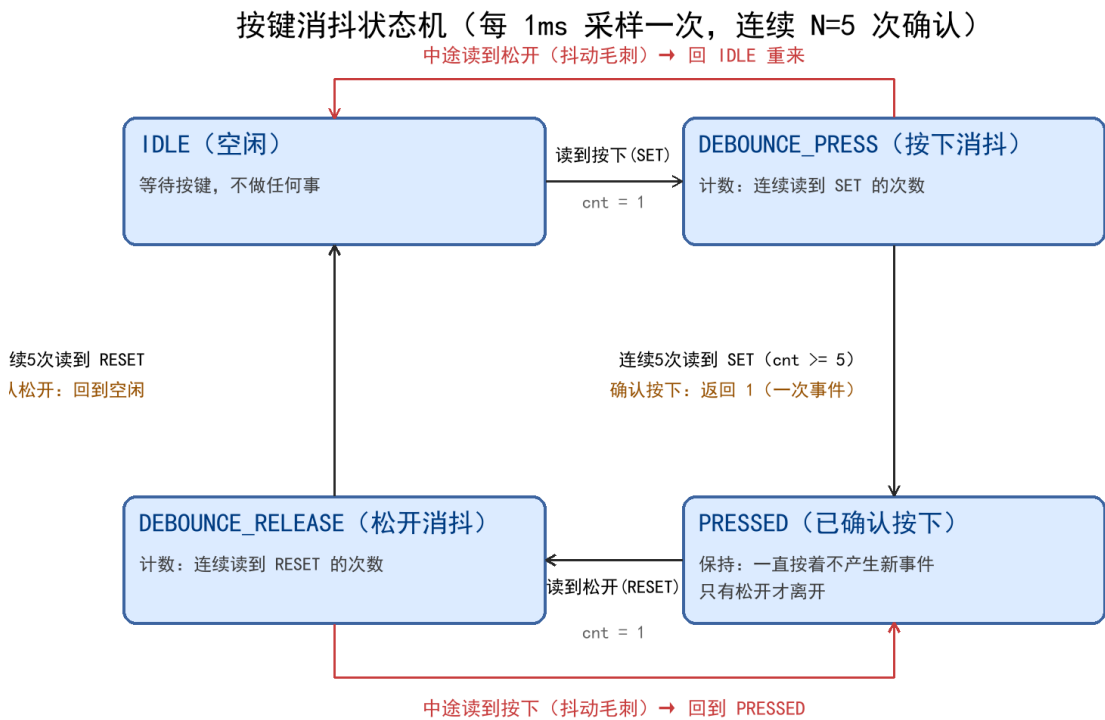


图 2 按键消抖状态机（每 1ms 采样一次，连续 N=5 次确认）

4.1 四个状态说明

状态	含义	进入条件
IDLE	空闲，等待按键	复位后 / 确认松开后

DEBOUNCE_PRESS	按下消抖，累计连续按下次数	在 IDLE 读到按下，cnt = 1
PRESSED	已确认按下，保持	连续 5 次读到按下，产生一次事件
DEBOUNCE_RELEASE	松开消抖，累计连续松开次数	在 PRESSED 读到松开，cnt = 1

4.2 完整代码（建议每 1ms 调用一次 Key_Scan）

```
typedef enum {
    KEY_IDLE, KEY_DEBOUNCE_PRESS, KEY_PRESSED, KEY_DEBOUNCE_RELEASE
} KeyState;

static KeyState key_state = KEY_IDLE;
static uint8_t debounce_cnt = 0;

#define KEY_DEBOUNCE_MAX    5           /* 连续 5 次采样，约 5ms */
#define KEY_PRESSED_LEVEL  GPIO_PIN_SET /* 按下 = 高电平 */

uint8_t Key_Scan(void)
{
    uint8_t level = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_3);
    switch (key_state)
    {
        case KEY_IDLE:
            if (level == KEY_PRESSED_LEVEL) /* 第一次看到按下，不急着信 */
            {
                debounce_cnt = 1;
                key_state = KEY_DEBOUNCE_PRESS;
            }
            break;

        case KEY_DEBOUNCE_PRESS:
            if (level == KEY_PRESSED_LEVEL)
            {
                if (++debounce_cnt >= KEY_DEBOUNCE_MAX) /* 连续 N 次都按下 */
                {
                    key_state = KEY_PRESSED;
                    return 1; /* 确认按下：产生一次事件 */
                }
            }
            else
            {
                key_state = KEY_IDLE; /* 抖动，回空闲重来 */
            }
            break;

        case KEY_PRESSED:
            if (level != KEY_PRESSED_LEVEL) /* 看到松开，进入松开消抖 */
            {
                debounce_cnt = 1;
                key_state = KEY_DEBOUNCE_RELEASE;
            }
            break;

        case KEY_DEBOUNCE_RELEASE:
            if (level != KEY_PRESSED_LEVEL)
            {
                if (++debounce_cnt >= KEY_DEBOUNCE_MAX)
```

```

        {
            key_state = KEY_IDLE;          /* 确认松开 */
        }
    }
    else
    {
        key_state = KEY_PRESSED;          /* 松开过程抖动，回到按下 */
    }
    break;
}
return 0;
}

```

4.3 一直按着时的逐次调用过程（每 1ms 一次）

第几次调用	进入状态	读到电平	发生的事	cnt
1	IDLE	按下	cnt=1, 进入按下消抖	1
2	DEBOUNCE_PRESS	按下	cnt++	2
3	DEBOUNCE_PRESS	按下	cnt++	3
4	DEBOUNCE_PRESS	按下	cnt++	4
5	DEBOUNCE_PRESS	按下	cnt++ $\geq 5 \rightarrow$ PRESSED, 返回 1	5
6...	PRESSED	按下	保持, 不产生新事件	5

4.4 关键理解（最容易绕晕的点）

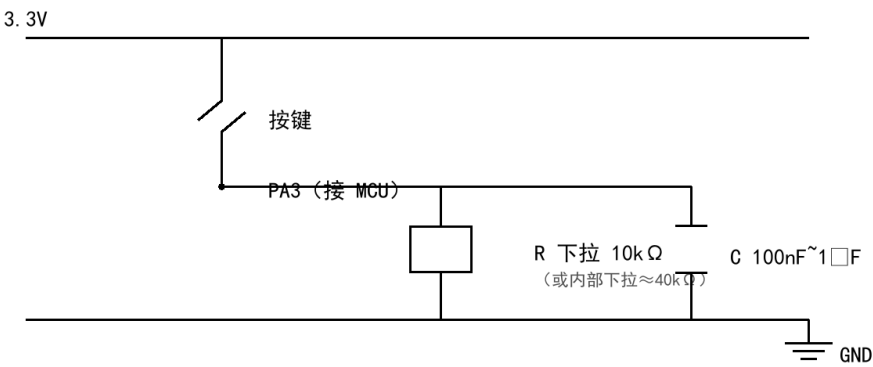
- ☐ Key_Scan() 每次调用只执行 switch 里的一个 case，不是全部。
- ☐ debounce_cnt = 1 只发生在「离开 IDLE」和「进入松开消抖」这两处；在 DEBOUNCE_PRESS 里只有 ++debounce_cnt 自增，不会重复赋 1。
- ☐ 一直按着时：IDLE $\rightarrow$ DEBOUNCE_PRESS (cnt 1 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ 4) $\rightarrow$ 第 5 次确认 $\rightarrow$ PRESSED，之后一直停在 PRESSED，不会来回跳。
- ☐ 状态机的本质：状态决定执行哪段代码；计数器在同一状态下反复累加；赋初值只在状态切换的那一次发生。

五、消抖方案三：硬件 RC 电容消抖

原理：电容两端电压不能突变，RC

构成低通滤波器，把按键的快速抖动「抹平」成缓慢变化的电平，MCU 读到的就是稳定电平。

RC 电容消抖电路（按键接 3.3V，高电平有效）



原理：电容电压不能突变，RC 低通滤波把抖动抹平。 $\tau = R \times C \approx 1\text{ms} \sim 10\text{ms}$

图 3 RC 电容消抖电路（按键接 3.3V、高电平有效的接法）

参数：时间常数 $\tau = R \times C$ 。10kΩ 下拉 + 100nF~1μF $\approx 1\text{ms} \sim 10\text{ms}$ ，正好覆盖 5~20ms 的抖动。电容别太大，否则按键响应会变迟钝。

优点：代码最简单，不需要软件消抖。缺点：要改硬件，且按键会「变钝」（有 RC 充放电延时）。

六、三种消抖方案对比

方案	是否阻塞	可靠性	代码量	适用场景
延时消抖	阻塞 20ms	二次确认	少	单个按键、简单工程
状态机消抖	不阻塞	连续 N 次确认	中	多按键、长按/双击、复杂工程
RC 电容消抖	不阻塞（硬件）	硬件滤波	最少	追求简单可靠、可改硬件

七、最终代码（key 工程 main.c 关键部分）

采用「延时 + 二次确认 + 松手消抖」，放在 CubeIDE 的 USER CODE 段内，重新生成不会被覆盖。

```
/* USER CODE BEGIN 2 */
uint8_t led_on = 0; /* LED 状态: 0=灭, 1=亮 */
/* USER CODE END 2 */

while (1)
{
    if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_3) == GPIO_PIN_SET) /* 第一次看到按下 */
    {
        HAL_Delay(20); /* 按下消抖 */
        if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_3) == GPIO_PIN_SET) /* 再次确认 */
        {
            led_on = !led_on; /* 翻转 */
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, led_on);
            while (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_3) == GPIO_PIN_SET) /* 等松手 */
            {
            }
            HAL_Delay(20); /* 松手消抖 */
        }
    }
}
```

八、补充知识点

8.1 三元运算符 ?:

格式：条件 ? 结果A : 结果B。条件成立取 A，否则取 B，是 if/else 的简写。

```
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7,  
                    led_on ? GPIO_PIN_SET : GPIO_PIN_RESET);  
  
// 等价于：  
// if (led_on) HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_SET);  
// else      HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_RESET);
```

8.2 GPIO_PIN_SET / GPIO_PIN_RESET 就是 1 / 0

HAL 里定义 #define GPIO_PIN_RESET 0u、GPIO_PIN_SET 1u。所以也可以直接写 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, led_on)，但用命名常量更清晰。

8.3 面包板结构

中间凹槽左右两排互不相通；同一边的 5 个孔上下连通。跨凹槽的接线必须用跳线或元件本身跨接。

8.4 STM32F103C8T6 的 PA12

PA12 复用功能是 USB_DP (USB D+)。Blue Pill 板上该引脚带 1.5kΩ 上拉到 3.3V，用来做 USB 全速设备识别。

8.5 状态机的核心理解

状态机的「状态」决定这次调用执行哪段代码；「计数器」在同一状态下反复累加；「赋初值」只在状态切换的那一次发生。理解这三点，状态机就不会绕晕。